

Basics

Array to DH doubly linked list

```
public static DoublyNode createList(Object[] arr) {  
    DoublyNode dh = new DoublyNode(elem: null, prev: null, next: null);  
    dh.next = dh;  
    dh.prev = dh;  
    DoublyNode tail = dh;  
    for (int i = 0; i < arr.length; i++) {  
        DoublyNode n = new DoublyNode(arr[i], tail, dh);  
        tail.next = n;  
        tail = n;  
        dh.prev = tail;  
    }  
    return dh;  
}
```

dh.next=dh;

dh.prev=dh;

Lab Task

Lab Task

In this part of the lab, we will play with Dummy Headed Doubly Circular Linked List. If you want to read about this type of linked list then check [this file](#).

Note: Even though we're learning Dummy Headed Doubly Circular Linked List, the methods you'll be writing simulate a different data structure. Can you guess what that is? Hint: It's something you face in front of the elevator everyday.

In the first part of this lab, you have to implement a waiting room management system in an emergency ward of a hospital. Your program will serve a patient on a **first-come, first-served basis**. The **Simulation of WRM** can be found [here](#).

Solve the above problem using a **Dummy Headed Doubly Circular Linked List**.

1. You need to have a **Patient** class so that you can create an instance of it (patient) by assigning id(integer), name (String), age (integer), and bloodGroup (String).
2. Write a **WRM** (waiting room management) class that will contain the following methods.
 - a. **registerPatient(id, name, age, bloodgroup):** This method will register a patient into your system. The method will create a Patient type object with the information received as a parameter. It means this method will add a patient-type object to your linked list.
 - b. **servePatient():** This method calls a patient to provide hospital service to him/her. In this method, you need to ensure that to serve the patient who was registered first. (serving means removing the patient from line)
 - c. **cancelAll():** This method cancels all appointments of the patients so that the doctor can go to lunch.
 - d. **canDoctorGoHome():** This method returns true if no one is waiting; otherwise, it returns false.
 - e. **showAllPatient():** This method prints all IDs of the waiting patients in sequential order. It means the patient who got registered first will come first, and so on.
 - f. **reverseTheLine():** This method reverses the patient line. It means the patient who got registered last will come first, and so on.

Important Hints:

1. In your program, **there won't be any class called "Node"**. Instead, the **Patient class** will work as the Node class.
2. On the other hand, the **WRM** class will act as the Dummy Headed Doubly Circular Linked List, which will contain all the other functions/methods.

`System.out.println();` -> `System.out.println()` function as

```
public class WRM {
    Patient dh;

    //The constructor is already created for you
    public WRM() {
        dh = new Patient(null, null, null, null, null, null);
        dh.next = dh;
        dh.prev = dh;
    }
}
```

```

public void registerPatient(int id, String name, int age, String bloodgroup) {
    Patient p= new Patient(id, name, age, bloodgroup, null, null);

    p.next=dh;
    p.prev=dh.prev;
    dh.prev.next=p;
    dh.prev=p;
}

```

```

public void servePatient() {
    if(dh.next!=dh){
        Patient first= dh.next;
        System.out.println("Serving "+ first.name+ " having id"+ first.id);
        dh.next=first.next;
        dh.next.prev=dh;
    }
}

```

```

public void showAllPatient() {
    if(dh.next==dh){
        System.out.println("No patient in the line");
    }

    else{
        Patient current=dh.next;
        System.out.println("The patient list is: ");

        while(current!=dh){
            System.out.println(current.name + " has the id "+ current.id);
            current=current.next;
        }
    }
}

```

```

public Boolean canDoctorGoHome() {
    if(dh.next==dh){

```

```
        System.out.println("The doctor can go home now");
        return true;
    }
    return false;
}
```

```
public void cancelAll() {
    dh.next=dh;
    dh.prev=dh;
    System.out.println("All appointments are cancelled");
}
```

//Doubly reverse

```
public void reverseTheLine() {
    Patient current=dh.next;
    Patient temp=null;

    while(current!=dh){
        temp=current.next;
        current.next=current.prev;
        current.prev=temp;

        current=temp;
    }

    temp = dh.next;
    dh.next = dh.prev;
    dh.prev = temp;

}
```

```
}
```

Lab Assignment

5. Sum Odd Append

Write a method/function called **sumOddAppend()**. The function will take only one parameter, referencing a **dummy-headed singly circular linked list**. The function/method removes all the nodes containing odd values while summing them up. Finally, a node containing the summation is inserted at the end.

NOTE:

1. YOU CAN CREATE ONLY 1 EXTRA NEW NODE using the given Node Class.

Given Dummy Headed Singly Circular Linked List	Expected Modified Linked List
	

//5

```
public static void sumOddAppend(Node dh) {
    if(dh.next==dh){
        return;
    }

    else{
        Node current= dh.next;
        Node prev=dh;
        int sum=0;

        while(current!=dh){
            int a=(int)current.elem;

            if((int)current.elem%2==0){
                prev=current;
                current=current.next;
            }

            else if(a%2!=0){
                sum+=a;
                prev.next=current.next; //prev isn't moving in this case
                current=current.next;
            }
        }

        Node newNode=new Node(sum);
        newNode.next=dh;
        prev.next= newNode;

        return;
    }
}
```



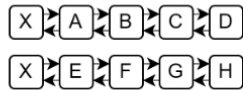
```
}
}
```

6. Pair Join

Write a method/function called **pairJoin()**. The function takes two parameters, referencing **two dummy heads** of two different Dummy Headed Doubly Linked Lists. The method/function will modify the connections of the two given linked lists and combine them in pairs as shown in the example below. The method/function will not return anything because the first dummy head will naturally become the new head.

NOTE:

1. YOU **CANNOT** CREATE ANY NEW NODES.
2. YOU CAN ASSUME THE LENGTH OF THE GIVEN TWO LISTS WILL ALWAYS BE EQUAL.

Given Dummy Headed Doubly Linked List	Expected Modified Linked List
	

```
//6
```

```
public static void pairJoin(DNode dh1, DNode dh2) {
    DNode current1=dh1.next;
    DNode current2=dh2.next;

    while(current1.next!=null){

        DNode temp1=current1.next;
        DNode temp2=current2.next;

        current1.next=current2;
        current2.prev=current1;

        current2.next=temp1;
        temp1.prev=current2;

        current1=temp1;
        current2=temp2;
    }
}
```

```

if(current2.next==null){
current1.next=current2;
current2.prev=current1;
current2.next=dh1;
dh1.prev=current2;
}

else{                                     //If 2nd linked list is longer
current1.next=current2;
current2.prev=current1;
while(current2.next!=null){
current2=current2.next;
}
current2.next=dh1;
dh1.prev=current2;
}
}

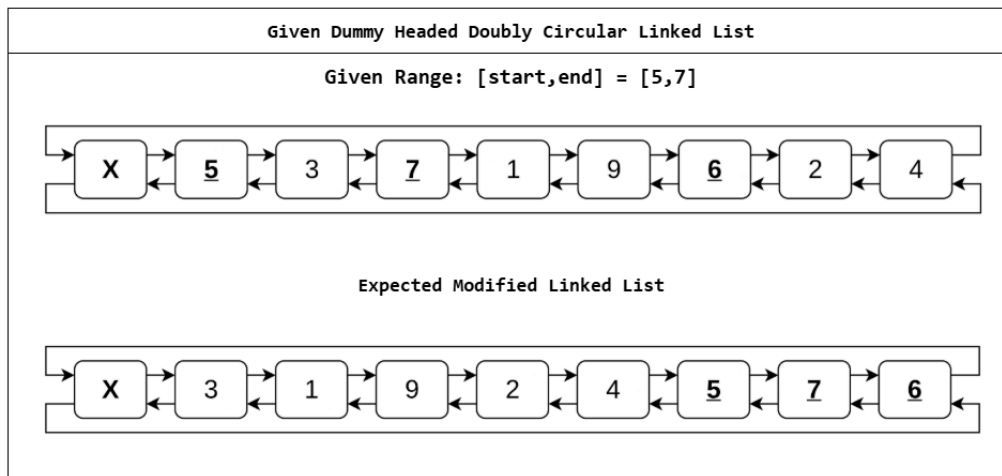
```

7. Range Move

Write a method/function called **rangeMove()**. The function takes three parameters, referencing **one dummy head and two integers**. The method/function will move all the nodes that fall within the given range of integers to the back of the linked list. The method/function will not return anything because the dummy head won't change.

NOTE:

3. YOU **CANNOT** CREATE ANY NEW LINKED LIST FOR THIS TASK
4. YOU **CANNOT** CREATE ANY EXTRA NODE.



//7

```
public static void rangeMove(DNode dh, int start, int end) {
    DNode prev=dh;
    DNode current=dh.next;
    DNode ogTail=dh.prev; //To find where to stop checking

    while(current!=ogTail){
        int a=(int)current.elem;

        if(a>=start && a<=end){ // else if hole ||
            DNode tail= dh.prev; //Locating the CURRENT TAIL
            DNode store=current;

            prev.next=current.next;
            current=current.next;
            current.prev=prev;

            tail.next=store;
            store.prev=tail;

            store.next=dh;
            dh.prev=store;
            tail=store;
        }

        else{
            current=current.next;
            prev=prev.next;
        }
    }
}
```

Lab Exam

1. Given an **even-lengthed doubly linked list** with **non-negative integers**, you have to solve the following problem inside a function or method:

Go through the linked list and find the **adjacent pair of nodes** with the **minimum sum**. **Replace** that pair with a new Node containing their **sum**; in this case, one node will get dropped.

NB: The **Node class** for the Doubly Linked list has already been **implemented**. The function or method you are creating will only take the **Head** of the DLL.

Sample Input	Explanation
15 ⇔ 10 ⇔ 21 ⇔ 9 ⇔ 5 ⇔ 20 ⇔ 9 ⇔ 10	Here, 9+5=14 is the smallest. So, you need to merge nodes 9 and 5, create a new node to store 14, and then place it there.
Sample Output	
15 ⇔ 10 ⇔ 21 ⇔ 14 ⇔ 20 ⇔ 9 ⇔ 10	15 ⇔ 10 ⇔ 21 ⇔ 14 ⇔ 20 ⇔ 9 ⇔ 10

```
public static DNode replaceMinPair(DNode head) {  
    if (head == null || head.next == null) return head;
```

```
    DNode current = head;  
    DNode minNode = head;  
    int minSum = head.elem + head.next.elem;
```

```
    while (current.next != null) {  
        int sum = current.elem + current.next.elem;  
        if (sum < minSum) {  
            minSum = sum;  
            minNode = current; //Store min node  
        }  
        current = current.next;  
    }
```

```
    DNode first = minNode;  
    DNode second = minNode.next;
```

```
    DNode newNode = new DNode(minSum, null, null);
```

```
    newNode.prev = first.prev;  
    newNode.next = second.next;
```

```
if (first.prev != null)
    first.prev.next = newNode;
else
    head = newNode;      // if first was head

if (second.next != null)
    second.next.prev = newNode;

return head;
}
```

RFTS Practice

You are given a **doubly linked list** (it is neither circular nor dummy-headed) and an integer k . Write a function named **rotate_doubly_linked_list()** that takes the head of the doubly linked list and an integer k as parameters. The function modifies the doubly linked list to be **rotated to the right** by k positions and returns the head.

```
public Node rotate_doubly_linked_list(Node head, int k) {
    if (head == null || k == 0) return head;

    // Find the length and the tail
    Node tail = head;
    int length = 1;
    while (tail.next != null) {
        tail = tail.next;
        length++;
    }

    k = k % length;
    if (k == 0) return head;

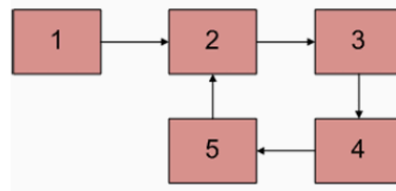
    // Move till the previous of k
    Node newTail = head;
    for (int i = 1; i < length - k; i++) {
        newTail = newTail.next;
    }

    Node newHead = newTail.next;

    newTail.next = null;
    newHead.prev = null;
    tail.next = head;
    head.prev = tail;

    return newHead;
}
```

Given a linked list, check if the linked list has a loop or not. The below diagram shows a linked list with a loop.



```
public boolean checkLoop(Node head) {  
    if (head == null) return false;  
  
    Node slow = head;  
    Node fast = head;  
  
    while (fast != null && fast.next != null) { //fast goes faster. So it can reach null faster  
        slow = slow.next;  
        fast = fast.next.next; //If the list has no loop, fast will reach null (the end).  
  
        if (slow == fast) {  
            return true;  
        }  
    }  
  
    return false;  
}
```

This process can be used to find first node of a cycle as well.

Palindrome linked list



Original list



Reversed list

```
public static boolean isPalindrome(Node head) {  
    if (head == null || head.next == null) return true;  
  
    // Step 1: reach middle  
    Node slow = head, fast = head;  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
    }  
  
    // Step 2: reverse second half  
    Node prev = null, curr = slow;  
    while (curr != null) {  
        Node next = curr.next;  
        curr.next = prev;  
        prev = curr;  
        curr = next;  
    }  
  
    // Step 3: Compare first half and reversed second half  
    Node left = head, right = prev;  
    while (right != null) {
```

```

        if (left.data != right.data) return false;
        left = left.next;
        right = right.next;
    }

    return true;
}

```

A teacher gives a child a set of toy boxes arranged in a circle, each placed in a specific position. As a challenge, the teacher asks the child to reverse the order of boxes between two specific positions m and n , while keeping the rest of the arrangement unchanged. Now, your task is to help the child fix the arrangement. The boxes are represented as a **Dummy Headed Circular Doubly Linked List**, where each node corresponds to a box.

You are given the **head** of the list and two integers m and n ($1 \leq m < n \leq \text{length of the list}$). **Reverse** the nodes from position m to n without affecting the rest of the list, and return the **modified list's head**. Consider that the **DoublyNode** class, **createdDLL()**, **printDLL()**, **nodeAt()** methods/functions are already provided.

Sample Function Call	Sample Returned Result
head => 10 ⇌ 40 ⇌ 30 ⇌ 20 ⇌ 50, $m = 2, n = 4$ reverseDLLBetweenTwoPositions(head, m, n)	Head of the modified linked list => 10 ⇌ 20 ⇌ 30 ⇌ 40 ⇌ 50

Note: Solve this task in **constant space complexity**, meaning you must modify the linked list **in-place** without creating a new one. m and n will never have the same value, and the linked list will always contain at least two nodes.

Sample Function Call	Sample Returned Result
head => 10 ⇌ 40 ⇌ 30 ⇌ 20 ⇌ 50, $m = 2, n = 4$ reverseDLLBetweenTwoPositions(head, m, n)	Head of the modified linked list => 10 ⇌ 20 ⇌ 30 ⇌ 40 ⇌ 50

```

public static DoublyNode reverseDLLBetweenTwoPositions(DoublyNode head, int m, int n) {
    if (head == null || head.next == head || m == n) return head;

```

//Find start

```

DoublyNode current = head.next;
for (int i = 1; i < m; i++) {
    current = current.next;

```

```

    }

    DoublyNode start = current;

    //Find end
    for (int i = m; i < n; i++) {
        current = current.next;
    }
    DoublyNode end = current;

    // Store borders
    DoublyNode beforeStart = start.prev;
    DoublyNode afterEnd = end.next;

    // Reverse between start and end
    DoublyNode temp = null;
    DoublyNode node = start;
    while (node != afterEnd) {
        temp = node.next;
        node.next = node.prev;
        node.prev = temp;
        node = temp;
    }

    // Reconnect the reversed part
    beforeStart.next = end;
    end.prev = beforeStart;

    start.next = afterEnd;
    afterEnd.prev = start;

    return head;
}

```